

Pesquisa aprofundada sobre *pivot_root* e *switch_root* em um Docker-like system from scratch

Arquitetura com control plane em Python, glue em Cython e data plane em C++

Escopo: troca correta da raiz do processo, limpeza de *oldroot*, `chdir("/")` e *safe unmount*. Data de elaboração: 12 mar. 2026.

A troca de raiz em um runtime de containers não deve ser tratada como um syscall isolado. Ela é uma transição de domínio com pré-condições de kernel, invariantes de segurança, telemetria, rollback e semântica explícita de limpeza.

Tese central. Para um projeto com metadados, API e política em Python, cola em Cython e execução nativa em C++, a melhor modelagem é uma capability de domínio chamada *Root Transition*, implementada no plano de dados por estratégias distintas. A estratégia padrão de produção deve ser a variante inspirada em *runc*, baseada em `pivot_root(".", ".")` e desmontagem via descritor de arquivo, enquanto uma estratégia clássica com diretório `put_old` pode servir como MVP pedagógico e ferramenta de depuração.

Conclusão operacional. *switch_root* é essencial para entender o problema, mas pertence primariamente ao mundo de initramfs e boot, não ao caminho feliz de um runtime de containers genérico. Em um sistema Docker-like, ele deve aparecer como modo restrito ou como inspiração para um fallback baseado em `MS_MOVE + chroot`, nunca como a primitiva principal do isolamento.

Resumo executivo

Este documento analisa, em profundidade, como implementar a feature de troca correta da raiz do processo em um runtime de containers construído do zero, com uma separação de responsabilidades muito próxima do que o ecossistema Docker acabou convergindo ao longo do tempo. A ideia de manter política, metadados, API e orquestração em Python, usando Cython como fronteira controlada e C++ como responsável pelos syscalls, pela conversa direta com o kernel e pela lógica sensível a descritores de arquivo, é sólida. O ponto crítico é reconhecer que *pivot_root*, *switch_root*, `chdir("/")` e o cleanup do *oldroot* não são detalhes locais. Eles definem a integridade do mount namespace e o grau real de isolamento do processo.^{[1][2][3][9][10]}

No Linux, *pivot_root* altera a raiz de um mount namespace e move a raiz antiga para um ponto especificado por `put_old`. A documentação do kernel impõe restrições que afetam diretamente o desenho do runtime: `new_root` precisa ser um mountpoint, `put_old` deve ser um diretório sob `new_root`, os mounts relevantes não podem estar em regime compartilhado e a operação exige privilégios compatíveis com `CAP_SYS_ADMIN` no namespace de mounts. Além disso, a chamada não garante que o diretório de trabalho atual do chamador passe automaticamente a apontar para a nova raiz; por isso, o passo seguinte correto é `chdir("/")`.^[1]

switch_root, por sua vez, é uma operação de espaço de usuário pensada para a transição do initramfs para o sistema de arquivos real no boot. Ela move mounts como `/proc`, `/sys`, `/dev` e `/run`, troca a raiz efetiva e remove recursivamente o antigo rootfs temporário. Essa semântica é útil para compreender limpeza e handoff de mounts, mas não deve ser transplantada, sem mediação, para um runtime de containers. O motivo é simples: o contrato de um container não é o mesmo contrato do processo `init` do sistema operacional.^{[2][10]}

Recomendação central

Modele a feature como um serviço nativo de *Root Transition* com três estratégias. A primeira, **PutOldPivotStrategy**, usa um diretório explícito sob o novo rootfs e é excelente para MVP, depuração e aprendizado. A segunda, **FDPIVOTStrategy**, deve ser a estratégia padrão de produção porque elimina a necessidade de criar um diretório `put_old` dentro do rootfs, suporta melhor cenários com raiz somente leitura e segue a prática consolidada no *runc*. A terceira, **MoveRootChrootStrategy**, reproduz um comportamento de *switch_root* apenas como fallback controlado para ambientes em que *pivot_root* não possa ser usado, com endurecimento adicional para `/proc` e `/sys`.^{[1][2][9][10]}

Do ponto de vista arquitetural, a melhor decisão é concentrar toda a sequência pós-fork em código nativo. O processo Python não deve, depois do fork, permanecer executando lógica da própria VM, do alocador ou do GIL enquanto o filho manipula mounts, namespaces, cwd e descritores. O papel do Python é compilar o plano declarativo; o papel do Cython é transportar DTOs estáveis e mapear erros; o papel do C++ é entrar no mount namespace, preparar a propagação, bind-montar o rootfs em si mesmo, executar a troca de raiz, desmontar o oldroot com segurança, montar os pseudo-filesystems previstos e, por fim, fazer `execve`.^{[5][8][9]}

Em termos de modelagem, o conceito de domínio correto não é “chamar *pivot_root*”, mas “realizar a transição de raiz do container”. Isso permite tratar estratégia, política de limpeza, snapshots de mountinfo, requisitos de observabilidade, rollback e compatibilidade com rootfs read-only como partes explícitas da linguagem do

sistema. Essa abordagem se encaixa bem em DDD, produz um design mais testável e torna a fronteira Python/Cython/C++ muito mais estável ao longo do tempo.

Sumário

1. Objetivo, escopo e tese de implementação
 2. Fundamentos conceituais: chroot, pivot_root, switch_root e mounts
 3. Contexto no Docker original, libcontainer, runc e OCI
 4. Tradução para o projeto Python + Cython + C++
 5. Proposta de System Design
 6. Proposta de Low Level Design
 7. Domain Driven Design aplicado à feature
 8. Padrões de design relevantes (GoF)
 9. Requirements funcionais e não funcionais
 10. Plano incremental de implementação
 11. Estratégia de testes e validação
 12. Conclusão
- Apêndice A. Sequências de syscall sugeridas
- Apêndice B. Interfaces indicativas entre Python, Cython e C++
- Referências

1. Objetivo, escopo e tese de implementação

O problema tratado aqui é específico e, ao mesmo tempo, estrutural: como trocar corretamente a raiz visível de um processo que será o processo init de um container, sem deixar a raiz antiga acidentalmente alcançável, sem quebrar a semântica de resolução de caminhos, sem vaziar eventos de mount para o host e sem misturar a lógica de orquestração em alto nível com a lógica de syscalls e descritores de arquivo em baixo nível. Em um projeto Docker-like feito do zero, esta feature fica exatamente na interseção entre isolamento de filesystem, lifecycle do container, montagem de pseudo-filesystems, segurança operacional e observabilidade de runtime.

O escopo deste relatório assume Linux contemporâneo com suporte a namespaces de mount, um runtime rootful como primeira entrega prática e um rootfs já materializado em disco a partir de imagem, overlay ou outro mecanismo equivalente. O foco não está em image distribution, cgroups, rede ou policy de segurança como seccomp, embora todos esses temas tangenciem a execução final do container. O foco está em uma capability muito particular do data plane: tomar posse de um mount namespace recém-preparado, fazer com que um diretório de rootfs se torne a nova raiz lógica do processo e garantir que, ao final da operação, a raiz antiga esteja desmontada ou pelo menos destacada de forma segura.

A tese deste documento é que a feature não deve ser implementada como um bloco de código ad hoc enfiado no caminho de start. Ela deve ser tratada como um subsistema explícito, com linguagem própria, estados observáveis e políticas configuráveis. A forma concreta dessa tese é a criação de um serviço nativo chamado *Root Transition Service*, chamado pelo plano de controle através de uma fronteira Cython estável, recebendo um *RuntimePlan* imutável e devolvendo um resultado estruturado com sucesso, falha, estágio, erro, duração, snapshots de mountinfo e decisão de estratégia.

Por que isso importa

No momento em que o runtime executa a troca de raiz, quase tudo o que pode dar errado é difícil de depurar depois: o processo pode já ter mudado de namespace, a visibilidade do filesystem muda, descritores podem apontar para árvores antigas e uma simples ausência de `chdir("/")` pode manter o *cwd* preso no oldroot. Tratar a operação como subsistema, e não como detalhe, é o que permite instrumentação séria, testes de regressão e decisões políticas como “strict unmount” versus “lazy detach”.

A outra tese central é a separação rígida entre intenção e execução. Em Docker, a evolução histórica foi justamente nessa direção: o daemon orquestra e persiste estado; a parte que conversa com as primitivas do kernel é empurrada para uma camada especializada. Quando o Docker abandonou o acoplamento a LXC e introduziu libcontainer, a troca de raiz, os mounts e os namespaces passaram a ser tratados por código nativo especializado, que depois evoluiu no mundo OCI e do *runc*.^{[6][7][8]} O seu projeto deve repetir essa lição desde o início.

2. Fundamentos conceituais: *chroot*, *pivot_root*, *switch_root* e mounts

2.1 O que *chroot* realmente faz

A chamada `chroot()` altera o diretório raiz usado pelo processo para resolver caminhos absolutos e essa raiz é herdada pelos filhos. Esse comportamento, por si só, já é útil para restringir a resolução de caminhos a uma subárvore do filesystem, mas ele não altera a estrutura do mount namespace e não move a raiz antiga para fora do caminho. Em outras palavras, *chroot* muda a interpretação de `"/"` pelo processo, mas não rearranja a topologia dos mounts da mesma forma que *pivot_root*.^[12]

Essa distinção explica por que *chroot* não é a primitiva principal de um runtime de containers. O container não precisa apenas “ver outro diretório como barra”; ele precisa trocar a raiz operacional do mount namespace num contexto que inclui mounts especiais, regras de propagação, política de cleanup e eventual readonly rootfs. Em alguns ambientes, um fallback baseado em `MS_MOVE + chroot` é útil, mas somente quando se reconhece explicitamente que se trata de uma segunda estratégia, com endurecimentos extras e semântica diferente da primária.

2.2 O que *pivot_root* faz e quais são suas pré-condições

A descrição canônica de *pivot_root* é a troca da raiz de um mount namespace. A operação recebe um `new_root` e um `put_old`; a raiz atual é movida para `put_old`, e o novo root passa a ocupar a posição lógica de `"/`". A utilidade disso para containers é evidente: o processo init do container passa a enxergar o rootfs preparado pelo runtime como a raiz real do seu namespace, enquanto a raiz antiga fica temporariamente estacionada em algum lugar que o runtime pode desmontar logo em seguida.^[1]

A importância prática está nas restrições impostas pelo kernel. `new_root` e `put_old` precisam ser diretórios; `put_old` precisa estar abaixo de `new_root`; `new_root` precisa ser um mountpoint; e os mounts envolvidos não podem estar em configuração compartilhada incompatível. A man page também explicita que a chamada exige privilégios administrativos adequados e alerta que o diretório de trabalho atual não é automaticamente “corrigido” para o novo root. O runtime deve, portanto, assumir que `chdir("/")` é parte integrante da feature, não um detalhe cosmético.^[1]

A exigência de que `new_root` seja um mountpoint é um detalhe que derruba implementações ingênuas. A solução mais comum e segura é fazer um bind mount do rootfs em si mesmo, operação que o transforma em mountpoint sem alterar sua aparência estrutural. O *runc* segue essa linha explicitamente, e vários materiais técnicos sobre mount namespaces apontam a mesma técnica como preparo necessário antes do *pivot_root*.^{[1][9]}
[11]

2.3 O papel de *switch_root*

switch_root é uma utilidade de boot, não um mecanismo genérico de containers. Sua função clássica é sair de um initramfs, mover mounts essenciais já existentes para o novo root, fazer com que esse root passe a ser a nova raiz do sistema e descartar o rootfs temporário antigo. A man page alerta que a operação move `/proc`, `/dev`, `/sys` e `/run` para o novo root e remove recursivamente o conteúdo da raiz antiga; o código do util-linux também restringe a limpeza destrutiva aos casos em que a raiz antiga é de fato um initramfs em `ramfs` ou `tmpfs`.^{[2][10]}

Essa semântica faz enorme sentido na sequência de boot do sistema operacional, porque o rootfs antigo existe apenas como trampolim para a raiz “real”. Já em um container, a raiz antiga normalmente é a raiz herdada do host na cópia inicial do mount namespace. Destruir recursivamente esse root é obviamente inaceitável, e mesmo mover mounts especiais só é seguro se eles foram criados especificamente para o namespace do container. Daí a regra prática: o conceito é útil, a semântica completa não.

Diferença de contrato

No boot, *switch_root* entrega o controle do sistema a um novo `init` e assume que o initramfs antigo precisa desaparecer. No runtime de containers, a raiz herdada do host não é um buffer descartável. O que se quer é tirá-la do caminho do processo do container, não tratá-la como mídia temporária do sistema todo.

2.4 Mount namespaces e propagação de mounts

Mount namespaces isolam a lista de mounts visível para um conjunto de processos. Quando um processo entra em um novo namespace de mounts, ele começa, em regra, com uma cópia da lista de mounts do namespace pai. Esse ponto é decisivo: o container não nasce em um vazio; ele herda uma topologia inicial que pode carregar regras de propagação compartilhada vindas do host, especialmente em sistemas geridos por systemd.^{[3][4][11]}

As regras de propagação definem como eventos de mount e unmount fluem entre “peers”. Em mounts *shared*, eventos podem propagar para mounts relacionados; em *private*, essa propagação deixa de existir; em *slave*, a relação se torna assimétrica, permitindo receber alterações do master sem replicá-las de volta. Esse vocabulário não é ornamental. Se o runtime ignorá-lo, o *pivot_root* pode falhar ou, pior, um bind mount de preparação do rootfs pode vazar de volta para o namespace pai.^{[3][4][9]}

A prática madura, observada no *runc*, separa duas decisões. A primeira é a política de propagação do root do namespace do container, muitas vezes `MS_SLAVE|MS_REC` por padrão, para evitar que alterações internas vazem para fora mas ainda permitir certa visibilidade unidirecional do host quando necessário. A segunda é uma normalização local do mount pai do rootfs, tornando-o privado antes do bind mount do rootfs em si mesmo. Essa dupla decisão merece existir como duas abstrações distintas no projeto, e não como um único “remount mágico”.^[9]

2.5 Oldroot cleanup, `chdir("/")` e unmount seguro

A feature só está completa quando a raiz antiga deixa de ser relevante para o processo. Fazer *pivot_root* sem cleanup adequado é trocar uma vulnerabilidade óbvia por um bug semântico mais sutil: o processo pode continuar com *cwd* preso no oldroot, algum descritor pode continuar apontando para ele, ou a própria árvore antiga pode permanecer montada dentro do novo rootfs em um caminho reservado. Por isso, a operação correta tem três atos inseparáveis: trocar a raiz, reposicionar o *cwd* para `/` e desmontar a raiz antiga com política explícita.^{[1][9]}

O termo “unmount seguro” precisa ser entendido com precisão. “Seguro” não significa apenas “funcionou”; significa “não deixou o oldroot alcançável para o processo do container e não produziu efeitos indesejados por propagação”. Em muitos runtimes de produção, o caminho escolhido é usar `umount2(..., MNT_DETACH)` depois de ajustar a propagação do oldroot para `MS_SLAVE|MS_REC`. Isso é pragmático porque evita que referências residuais triviais provoquem erro imediato. Para depuração e testes, entretanto, é saudável oferecer também um modo estrito, que tente primeiro um unmount normal e só faça fallback para lazy detach quando a política permitir.

Antes e depois do root switch

O objetivo não é apenas trocar o ponto de vista de "/", mas remover a alcançabilidade prática do oldroot.

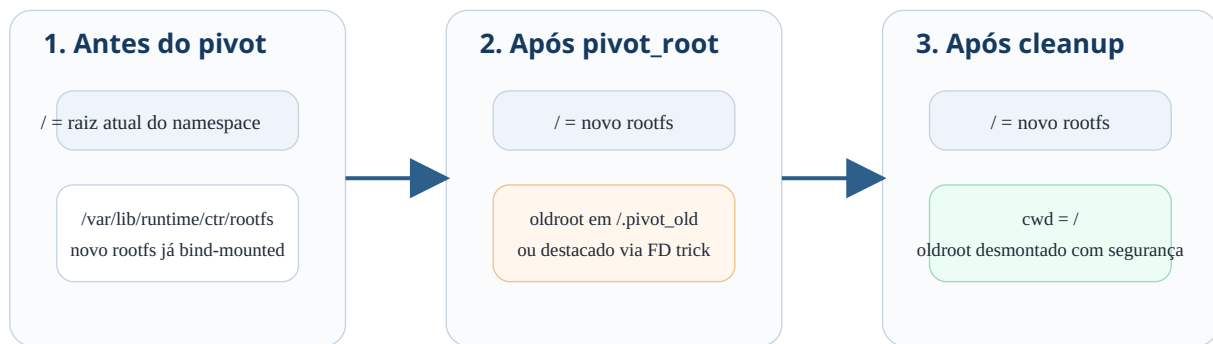


Figura 1. A feature termina apenas quando a nova raiz assume o papel de "/", o `cwd` passa a ser "/" e o oldroot deixa de ser alcançável.

Há duas famílias principais de cleanup. A família clássica usa um diretório explícito como `/.pivot_old`, faz o `pivot_root`, muda para `/`, desmonta esse caminho e remove o diretório. A família moderna, popularizada pelo `runc`, usa o truque documentado pela própria `man` page: entrar no novo root, chamar `pivot_root(".", ".")`, voltar ao oldroot por descritor de arquivo, marcar a raiz antiga como `slave` e desmontar `"."` com `MNT_DETACH`. A principal vantagem é não precisar criar um diretório extra dentro do rootfs, o que é especialmente valioso quando a raiz final é `readonly`.^{[1][9]}

Primitiva	Definição operacional	Quando faz sentido	Risco principal	Recomendação no projeto
<code>chroot()</code>	Troca a raiz usada na resolução de caminhos absolutos do processo.	Fallback controlado, ambientes limitados, fase final de um fluxo <code>MS_MOVE</code> .	Não rearranja o mount namespace nem resolve oldroot por si só.	Não usar como estratégia principal.
<code>pivot_root(new, put_old)</code>	Troca a raiz do mount namespace e move a raiz antiga para <code>put_old</code> .	MVP claro, rootfs gravável, depuração pedagógica.	Exige diretório reservado no rootfs e disciplina de cleanup.	Ótimo para primeira implementação.
<code>pivot_root(".", ".")</code>	Variante por descritor de arquivo que evita criar <code>put_old</code> explícito.	Produção, <code>readonly</code> rootfs, aderência a prática do <code>runc</code> .	Exige maior cuidado com FDs, <code>cwd</code> e ordem de operação.	Deve virar padrão de produção.
<code>switch_root</code>	Fluxo de boot que move mounts essenciais, troca a raiz e apaga <code>initramfs</code> .	Boot, <code>initramfs</code> , handoff para o <code>init</code> real.	Semântica destrutiva e inadequada para container genérico.	Usar apenas como referência conceitual.

Primitiva	Definição operacional	Quando faz sentido	Risco principal	Recomendação no projeto
<code>MS_MOVE + chroot</code>	Move a árvore de mounts do novo root para <code>/</code> e depois faz <code>chroot</code> .	Fallback quando <code>pivot_root</code> não é viável.	Exige mascarar ou desmontar mounts perigosos fora do rootfs antes do <code>chroot</code> .	Implementar como fallback endurecido.

3. Contexto no Docker original, libcontainer, *runc* e OCI

Entender como o Docker lidou com esse tema ajuda a evitar decisões arquiteturais ruins. Nas primeiras versões, o Docker se apoiava em LXC como backend de execução. A grande inflexão veio na era do Docker 0.9, quando o projeto introduziu libcontainer como execution driver nativo, deixando de depender de uma camada externa para acessar as APIs de containers do kernel. A partir dali, a topologia conceitual ficou mais nítida: o daemon administra intenção e estado; a camada nativa prepara namespaces, mounts, cgroups e processo init.^{[6][7]}

No ecossistema OCI, essa separação amadureceu ainda mais. A especificação de runtime descreve o root filesystem, a ordem dos mounts e a posição de hooks no ciclo de vida. Em particular, hooks como `createRuntime` e `createContainer` são pensados para ocorrer antes do `pivot_root` ou de um equivalente, o que mostra que o problema da troca de raiz faz parte do core runtime path, não de uma conveniência periférica.^[5]

O *runc*, herdeiro direto da linhagem libcontainer, é o melhor espelho técnico para o seu projeto. O código de preparação do rootfs faz três coisas particularmente instrutivas. Primeiro, define a política de propagação do `/` do namespace, tipicamente como `MS_SLAVE|MS_REC` ou conforme a configuração. Segundo, garante que o mount pai do rootfs não permaneça compartilhado, porque isso faria o `pivot_root` falhar e poderia vaziar bind mounts de preparação para o namespace pai. Terceiro, bind-mounta o rootfs em si mesmo para garantir que ele seja mountpoint. Só então executa a estratégia de troca de raiz.^[9]

O detalhe mais revelador aparece na implementação de `pivotRoot()` do *runc*. Em vez de criar um diretório `put_old` dentro do rootfs, o código abre descritores para a raiz antiga e para o novo root, muda o `cwd` para o novo root, chama `pivot_root(".", ".")`, retorna ao oldroot por descritor, remonta-o como *slave*, desmonta `."` com `MNT_DETACH` e só então faz `chdir("/")`. Essa escolha não é apenas elegante; ela evita criar diretórios auxiliares no rootfs e funciona bem quando a raiz final deve ser somente leitura.^{[1][9]}

Lição importada do runc

A estratégia por descritor de arquivo transforma a limpeza do oldroot em um protocolo robusto, não dependente da mutabilidade do rootfs. Para um runtime construído do zero, isso é o caminho mais próximo do estado da prática.

O próprio *runc* mostra, contudo, que nem sempre *pivot_root* pode ser usado. Existe um caminho alternativo, conhecido no código como `msMoveRoot`, em que o rootfs é movido para `/` via `MS_MOVE` e um `chroot` finaliza a nova visão de caminhos. O ponto crucial é o endurecimento de segurança que antecede esse fluxo: mounts completos de `/proc` e `/sys` que permaneceriam alcançáveis fora do rootfs são mascarados ou destacados para evitar que o processo do container retenha acesso indevido a pseudo-filesystems do host.^[9]

É justamente aqui que a diferença entre *switch_root* e runtime de containers fica cristalina. O mundo do boot supõe mounts especiais pré-existentes que devem ser movidos para o novo root. O mundo do container supõe que esses mounts precisam ser intencionalmente planejados dentro do novo root do container, muitas vezes como novas instâncias montadas depois do *pivot_root*. Portanto, em vez de “implementar *switch_root*”, o projeto deveria “implementar um modo *move-root/chroot* inspirado nas lições de *switch_root* e *runc*”.

Outro ponto importante da especificação OCI é a modelagem do rootfs como entidade declarativa. `root.path` aponta para um diretório existente e os mounts são aplicados em ordem definida. Isso sugere uma diretriz de design útil: o Python deve compilar a intenção em um plano ordenado de mounts, capacidades e política de root switch, enquanto o C++ executa esse plano como sequência determinística, emitindo eventos em cada estágio. Esse arranjo aproxima o projeto da clareza conceitual de containerd + runc sem exigir que sua implementação replique exatamente a arquitetura desses projetos.^{[5][8]}

4. Tradução para o projeto Python + Cython + C++

A proposta de divisão tecnológica do projeto é apropriada desde que a fronteira entre as linguagens seja desenhada com dureza. Python deve concentrar tudo o que diz respeito a intenção, política, metadados e APIs. Isso inclui a modelagem do *ContainerSpec*, a persistência da decisão de estratégia, a compilação do plano de mounts, a auditoria, o agendamento e a exposição de endpoints ou comandos administrativos. Cython deve ser apenas uma camada anti-corrupção entre a linguagem ubíqua do domínio em Python e as estruturas estáveis do runtime em C++. C++ deve assumir o fluxo sensível a processos, threads, descritores, namespaces, syscalls e rollback local.

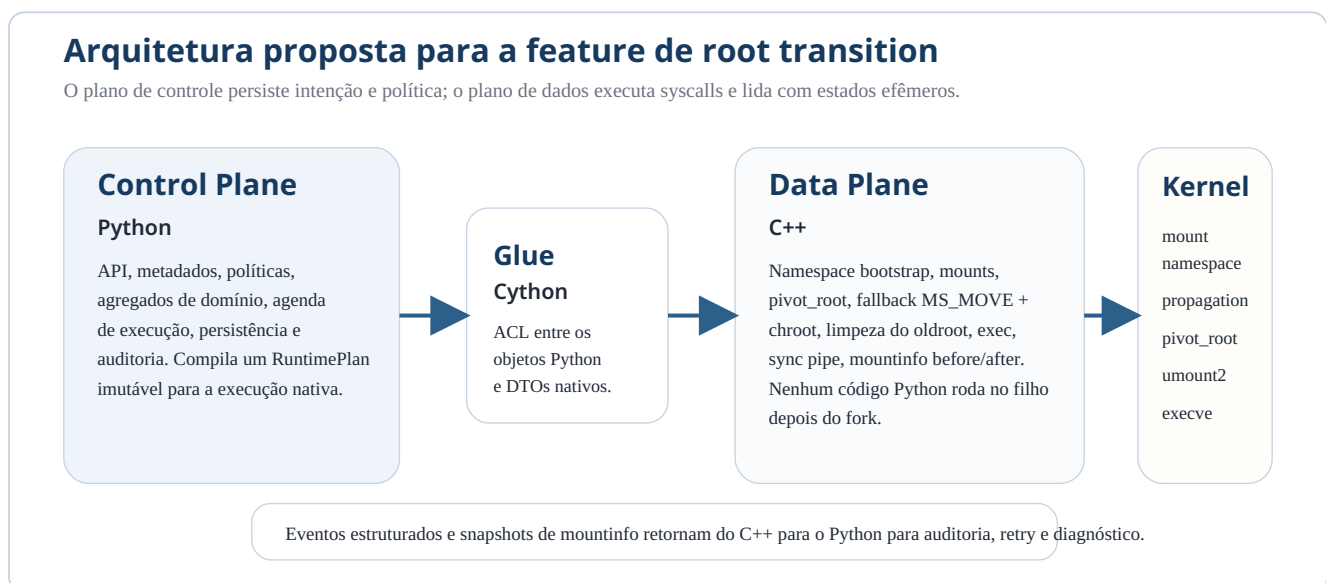


Figura 2. A cola Cython deve permanecer estreita; o caminho pós-fork pertence integralmente ao código nativo.

Essa divisão não é mera preferência estilística. A manipulação de namespaces e mounts depois de um fork é um lugar especialmente ruim para deixar a VM de Python, o GIL e callbacks dinâmicos aparecerem. A recomendação forte é que, uma vez iniciado o worker nativo, o caminho crítico siga inteiramente em C++ até o `execve` do init do container ou até uma falha que seja devolvida ao processo pai por um pipe de sincronização. Python participa antes, ao compilar o plano, e depois, ao receber o resultado e persistir eventos. No meio, ele não deve existir.

Em termos organizacionais, vale pensar o projeto em duas metades. A metade declarativa é um sistema de decisão. Ela sabe que o container A quer rootfs B, readonly true, root propagation slave, cleanup lazy, mounts padrão X e estratégia preferencial Y. A metade executora é um sistema de compromisso com o kernel. Ela sabe abrir o diretório certo com `O_PATH` ou equivalente, validar se é mountpoint, normalizar a propagação, executar a sequência de syscalls na ordem correta, fechar descritores com `CLOEXEC` e relatar exatamente onde e por que falhou. Misturar essas duas metades produz código frágil e difícil de recuperar após falha.

Recomendação de fronteira

A chamada Python -> Cython -> C++ deve transportar um DTO imutável, sem callbacks, sem objetos Python vivos no lado nativo e sem lógica de negócio escondida no Cython. O Cython deve traduzir estruturas e liberar o GIL quando apropriado; a semântica da feature pertence a Python e C++, não ao meio do caminho.

Há ainda uma nuance importante: a API Python não precisa chamar diretamente o código que fará `pivot_root`. Uma alternativa arquitetural madura é que ela acione um worker C++ dedicado, possivelmente um processo auxiliar do próprio runtime. Isso preserva o papel do Cython como glue para validações, consultas e operações mais simples, mas isola a sequência crítica em um binário nativo. Caso se prefira manter tudo em uma biblioteca C++ carregada por Cython, o requisito continua o mesmo: o filho pós-fork não deve executar de volta no interpretador.

A melhor forma de encapsular a feature é nomear explicitamente a decisão. Em vez de flags soltas como `no_pivot` ou `cleanup_lazy`, a API deveria trabalhar com um objeto de política, algo como *RootSwitchPolicy*, contendo modo preferido, fallback permitido, política de unmount, política de propagação, expectativa de readonly rootfs e regras para pseudo-filesystems padrão. Esse objeto é muito mais rico semanticamente e conversa melhor com DDD, versionamento de API e auditoria de runtime.

5. Proposta de System Design

Do ponto de vista de System Design, a feature deve ser pensada como parte do ciclo de vida de *StartContainer*, mas com identidade própria dentro desse ciclo. O fluxo de alto nível começa quando o usuário ou outro serviço submete uma intenção de execução. O plano de controle valida o *ContainerSpec*, resolve a imagem em um rootfs materializado, compila um *RuntimePlan* ordenado e persiste o estado “ReadyToStart”. Em seguida, um worker nativo é acionado com esse plano. A partir desse ponto, o controle muda de natureza: o que antes era desejo declarativo passa a ser manipulação concreta do mount namespace.

Sequência recomendada de start do container

A troca de root acontece no worker nativo, após a preparação do mount namespace e antes do exec do processo init.

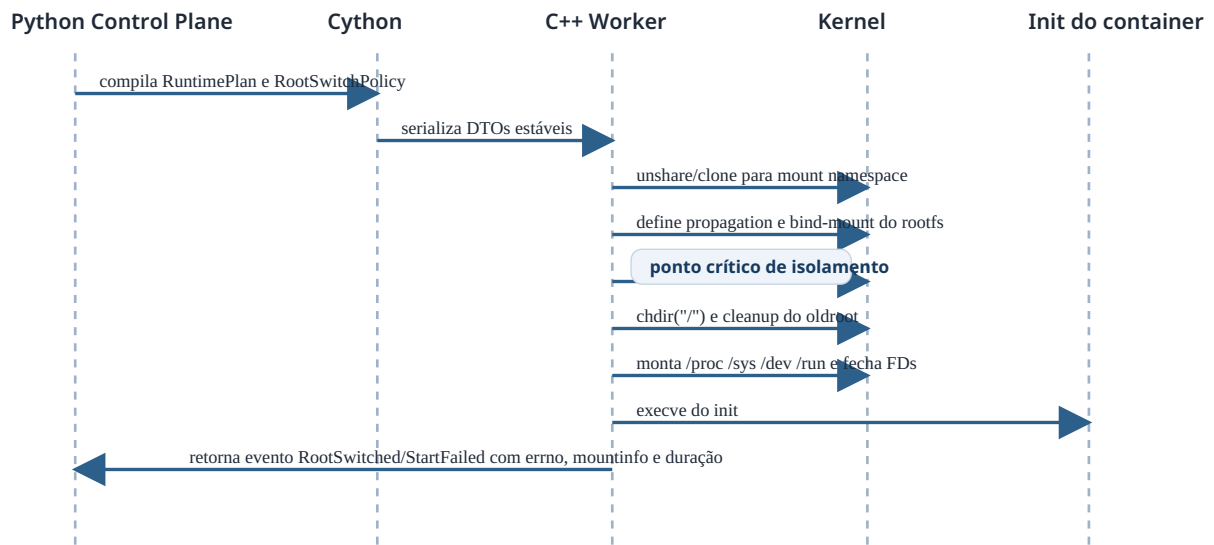


Figura 3. O root switch é uma etapa delimitada do start, mas com retorno observável e persistente para o plano de controle.

Uma forma útil de visualizar os componentes é a seguinte. O **API and Metadata Service**, em Python, recebe comandos, persiste specs, políticas e eventos de negócio. O **Runtime Orchestrator**, também em Python, compila um *RuntimePlan* a partir dos agregados do domínio e solicita execução ao data plane. O **Cython Bridge** converte esse plano em estruturas nativas, cuidando de ABI, strings, enums e códigos de erro. O **Runtime Engine**, em C++, cria ou entra no namespace, prepara mounts e executa a sequência de root transition. Um **Event Sink** persistente, novamente do lado Python, registra o resultado detalhado da execução.

A decisão mais importante desse design é que o root switch vira uma fase nomeada do start, com entrada e saída explícitas. Isso permite que o worker reporte eventos como *RootfsPrepared*, *MountPropagationNormalized*, *RootTransitionStarted*, *RootTransitionCompleted*, *OldRootUnmounted* e *InitExeced*. Quando a execução falha, o sistema não registra apenas “container failed to start”, mas sim “failed during ROOT_TRANSITION at stage PIVOT_ROOT with errno EINVAL and mountinfo snapshot attached”. Essa diferença de granularidade vale ouro em um sistema de containers.

O estado persistido do container deve distinguir pelo menos intenção, preparação e execução. Um desenho enxuto, mas robusto, teria estados semelhantes a *SpecCreated*, *RootfsStaged*, *RuntimePlanCompiled*, *RootTransitionPending*, *RootTransitionApplied*, *InitSpawned*, *Running*, *Exited* e *Failed*. O evento *RootTransitionApplied* não significa ainda “container running”; ele significa que a raiz já foi trocada e o processo está pronto para entrar na fase de mounts padrão e `exec`.

Decisão de responsabilidade

O plano de controle escolhe a política; o plano de dados escolhe apenas o melhor caminho técnico compatível com a política e com as capacidades reais do kernel. Se o plano de controle exigir estritamente `pivot_root`, o data plane não deve fazer fallback silencioso para `MS_MOVE + chroot`. Se o plano de controle permitir fallback, o data plane pode decidir com base em detecção de ambiente e devolver no resultado a estratégia efetivamente usada.

O sistema também precisa de um canal de erro síncrono entre o worker nativo e o processo pai. Esse canal pode ser um pipe simples com mensagens estruturadas ou um protocolo binário mínimo. O que importa é que, entre a criação do processo filho e o `execve` final, qualquer falha em `mount`, `pivot_root`, `chdir`, `umount2` ou montagem de `/proc` seja serializada em formato estável e entregue ao pai antes do encerramento. Isso impede que o plano de controle dependa apenas de exit codes opacos.

Vale ainda distinguir o *RuntimePlan* do *ExecutionAttempt*. O primeiro é declarativo e pode ser reusado ou auditado. O segundo representa a materialização concreta de uma tentativa, com timestamps, PIDs, namespace IDs, estratégia escolhida, mountinfo antes e depois, e eventual erro. Essa separação produz um modelo muito mais adequado para retries e para investigação de incidentes.

6. Proposta de Low Level Design

6.1 Invariantes e pré-validações

O Low Level Design começa antes do primeiro syscall de root switch. O runtime precisa validar o caminho do rootfs por descritor de arquivo, idealmente evitando resoluções frágeis por string quando o kernel suportar APIs como `openat2`. Precisa verificar que o rootfs existe, é diretório e está materializado da forma esperada. Precisa também verificar ou forçar que o rootfs se torne mountpoint, normalmente por bind mount recursivo em si mesmo, e normalizar a propagação do mount pai para um modo compatível com a operação. Essas checagens devem ocorrer no lado nativo, mesmo que o Python já tenha validado semanticamente o plano.

Outro invariante é a higiene de descritores. Os FDs usados para navegar entre oldroot e newroot precisam ser abertos com `CLOEXEC` para não vazarem para o processo do container após o `execve`. O mesmo vale para pipes de sincronização, eventfds ou qualquer descritor usado pela infraestrutura. Em sistemas de containers, bugs de vazamento de FD costumam se disfarçar como bugs de limpeza de oldroot ou de disponibilidade de mount, porque um descritor aberto pode manter referências inesperadas.

O runtime também precisa decidir explicitamente o que fazer com mounts padrão. Uma prática sólida é não misturar a troca de raiz com a criação de `/proc`, `/sys`, `/dev` e `/run`. A troca de raiz produz um namespace com nova barra. Depois disso, um *StandardMountPlan* monta ou move pseudo-filesystems de acordo com a política do container. Essa separação reduz o acoplamento e facilita testes direcionados.

Pré-condição	Racional técnico	Mitigação recomendada
Rootfs é diretório existente	Evita falhas triviais e diferencia erro de staging de erro de runtime.	Validar por FD no C++ e registrar erro de domínio distinto.
Rootfs é mountpoint	<i>pivot_root</i> exige <code>new_root</code> como mountpoint.	Aplicar <code>mount(rootfs, rootfs, ..., MS_BIND MS_REC, ...)</code> .
Mount pai do rootfs não é shared	Bind mount e <i>pivot_root</i> podem falhar ou vaziar para fora.	Subir até encontrar o mount pai e remountar como private.
CWD final deve ser <code>/</code>	Evita manter o processo referenciando oldroot após a transição.	Executar <code>chdir("/")</code> como etapa obrigatória.
Oldroot deve deixar de ser alcançável	Sem cleanup a feature está incompleta e o isolamento é ambíguo.	Aplicar unmount estrito ou lazy detach conforme política.

6.2 Estratégia 1: *PutOldPivotStrategy*

A primeira estratégia é a mais didática: criar um diretório reservado dentro do rootfs, por exemplo `/.pivot_old`, executar `pivot_root(new_root, put_old)`, mudar o diretório de trabalho para `"/`, desmontar `/.pivot_old` e remover o diretório. Ela tem a vantagem da clareza. Em logs, em testes e em depuração manual, o fluxo é intuitivo. Também torna mais simples verificar o efeito da troca de raiz olhando a árvore de mounts no meio do processo.

Apesar dessa clareza, a estratégia tem uma limitação estrutural: ela exige um caminho reservado dentro do novo root. Se o rootfs final deve ser estritamente somente leitura, esse diretório precisa existir antes, o que polui a imagem ou obriga staging adicional. Além disso, o runtime precisa garantir que nenhum mount do usuário ou do próprio sistema seja permitido nesse caminho reservado. Ainda assim, como MVP, é excelente. Ele força a equipe a compreender as invariantes da operação e a construir a telemetria correta antes de adotar o caminho mais compacto.

```
// Sequência simplificada da PutOldPivotStrategy
make_root_propagation_safe();
ensure_rootfs_is_mountpoint(rootfs);
create_reserved_dir(rootfs, "/.pivot_old");
chdir(rootfs);
pivot_root(".", "/.pivot_old");
chdir("/");
remount("/.pivot_old", MS_SLAVE | MS_REC);
umount2("/.pivot_old", cleanup_mode == LAZY ? MNT_DETACH : 0);
rmdir("/.pivot_old");
```

6.3 Estratégia 2: *FDPivotStrategy* inspirada em *runc*

A segunda estratégia é a recomendada para produção. Em vez de criar `put_old` explícito, ela abre descritores para a raiz antiga e para o novo rootfs, muda o diretório de trabalho para o novo root, executa

`pivot_root(".", ".")`, navega de volta ao oldroot usando o descritor antigo, marca a raiz antiga como *slave*, desmonta `"."` com `MNT_DETACH` e então faz `chdir("/")`. A própria man page documenta esse uso, e o *runc* o adota exatamente para evitar a necessidade de criar diretórios auxiliares dentro do rootfs.^{[1][9]}

O ganho prático é enorme. Primeiro, a estratégia funciona melhor com rootfs readonly, um requisito muito comum em ambientes endurecidos. Segundo, ela reduz a superfície de acoplamento com o conteúdo da imagem. Terceiro, ela expressa melhor o que está acontecendo: o runtime usa FDs estáveis para referenciar as duas árvores e faz o cleanup sem depender de nomes de caminho persistentes. Em um projeto escrito em C++, isso casa muito bem com RAII, wrappers de descritor e protocolos determinísticos de cleanup.

```
// Sequência simplificada da FDPivotStrategy
oldroot_fd = open("/", O_DIRECTORY | O_RDONLY | O_CLOEXEC);
newroot_fd = open(rootfs, O_DIRECTORY | O_RDONLY | O_CLOEXEC);
fchdir(newroot_fd);
pivot_root(".", ".");
fchdir(oldroot_fd);
mount("", ".", "", MS_SLAVE | MS_REC, "");
umount2(".", MNT_DETACH);
chdir("/");
```

Insight de design

A escolha por descritor de arquivo é mais do que um truque de syscall. Ela indica uma filosofia de implementação: as referências importantes do runtime devem ser mantidas por FD sempre que possível, porque FDs sobrevivem melhor a trocas de cwd, reduzem ambiguidades com symlinks e deixam a limpeza menos dependente de strings de caminho.

6.4 Estratégia 3: *MoveRootChrootStrategy* como fallback endurecido

A terceira estratégia existe para lidar com ambientes em que *pivot_root* não possa ou não deva ser usado. Nela, o runtime move o rootfs para `/` com `MS_MOVE` e depois executa um `chroot(".")` seguido de `chdir("/")`. O problema é que isso não fornece automaticamente o mesmo contrato de segurança. Se mounts completos de `/proc` ou `/sys` do host permanecerem alcançáveis fora da nova árvore, o container pode ver mais do que deveria. O *runc* explicita esse risco e endurece o fluxo antes da troca.^[9]

No seu projeto, essa estratégia deve ser declaradamente um fallback e vir acompanhada de uma verificação mais agressiva dos mounts residuais. Em particular, o worker nativo precisa detectar full procfs e sysfs mounts fora do rootfs e desmontá-los ou mascará-los antes do `chroot`. Também é recomendável montar novas instâncias de `/proc` e `/sys` somente depois de estabelecida a nova barra. Esse custo adicional é o preço de tratar o fallback como sério, e não como gambiarra.

6.5 StandardMountPlan e separação de fases

Depois da troca de raiz, o runtime entra em outra fase: a construção do ambiente padrão do container. Essa fase deve ser independente da estratégia de root switch usada. Ela monta `/proc`, `/sys`, `/dev`, `/run`, aplica readonly ou bind mounts adicionais, materializa `/etc/resolv.conf` ou equivalentes e finalmente ajusta

usuário, grupos, capabilities e executável inicial. Essa separação é importante porque um bug no root switch não deve ser confundido com bug em mount de pseudo-filesystem.

6.6 Modelo de erros, rollback local e telemetria

O modelo de erro ideal para esse subsistema é estruturado. Toda operação sensível deve registrar estágio lógico, syscall, argumentos relevantes, erro, mensagem amigável e, quando possível, snapshots de `/proc/self/mountinfo` antes e depois. Quando a falha ocorre antes do `pivot_root`, o worker pode simplesmente desmontar o que montou e sair. Quando ocorre depois do `pivot_root` mas antes do `execve`, o processo filho ainda precisa reportar com precisão o ponto da falha ao pai e tentar uma limpeza mínima segura antes de abortar.

O ponto essencial é que rollback aqui nunca será perfeito. Depois que a raiz do processo já mudou, desfazer tudo como se nada tivesse acontecido nem sempre é viável ou desejável. Por isso, a meta do rollback local é outra: impedir que um processo semi-inicializado continue vivo em estado inconsistente e garantir que o plano de controle receba informação suficiente para classificar e limpar o incidente.

7. Domain Driven Design aplicado à feature

A feature pede um modelo de domínio explícito porque cruza regras de negócio operacionais, capacidades do kernel e observabilidade. Um erro recorrente em runtimes jovens é tratar tudo isso como biblioteca utilitária. Em DDD, a melhor leitura é que o sistema possui ao menos quatro bounded contexts relevantes. O primeiro é **Container Lifecycle**, que representa estados, comandos e eventos de start, stop e cleanup. O segundo é **Filesystem Isolation**, que representa rootfs, mounts, políticas de root switch, cleanup e pseudo-filesystems. O terceiro é **Runtime Execution**, que lida com workers, tentativas, sincronização e telemetria. O quarto é **Metadata and Audit**, que persiste intenção, eventos e resultados observáveis.

Dentro desse desenho, a entidade central do lado declarativo pode ser o agregado *Container*, contendo referências a *ContainerSpec*, *RootfsReference*, *IsolationPolicy* e *ExecutionHistory*. O objeto de valor mais importante para esta feature seria *RootSwitchPolicy*, com campos como modo preferido, fallback permitido, política de unmount, propagação solicitada, readonly rootfs e mounts padrão esperados. Como objeto de valor, ele deve ser imutável, comparável e serializável sem ambiguidade.

Do lado de serviços de domínio, vale separar um *RuntimePlanCompiler* de um *RootTransitionDomainService*. O compilador recebe a intenção declarativa do container e produz um *RuntimePlan* ordenado. O serviço de root transition recebe esse plano e decide qual estratégia é semanticamente válida. Note a palavra “semanticamente”: a escolha entre *PutOldPivotStrategy* e *FDPivotStrategy* não é feita porque uma classe “gosta mais” da outra, mas porque o domínio sabe que um rootfs readonly torna a estratégia por descritor mais apropriada. A execução física continua no C++, mas a decisão pode e deve ser representada no domínio.

Cython como anti-corruption layer

No vocabulário de DDD, Cython deve atuar como uma ACL literal. Ele impede que estruturas internas do runtime nativo contaminem a linguagem do domínio em Python e, ao mesmo tempo, evita que objetos ricos do domínio cruzem para o lado nativo de forma opaca. A fronteira ideal é feita de DTOs explícitos, enums estáveis e mensagens de erro estruturadas.

Os eventos de domínio são especialmente valiosos aqui. Em vez de tratar o start como ato monolítico, o sistema pode emitir *RootTransitionRequested*, *RootTransitionSelected*, *RootTransitionStarted*, *RootTransitionCompleted* e *RootTransitionFailed*. Um evento como *OldRootUnmounted* pode parecer excessivo no início, mas se torna precioso quando o time precisa demonstrar auditabilidade ou construir dashboards de taxa de fallback versus caminho primário.

Há ainda um aspecto de saga. O comando *StartContainer* é naturalmente uma saga porque atravessa persistência, staging de rootfs, execução nativa e processo de longa duração. O passo de root switch é um subpasso da saga, com compensações imperfeitas, e merece sua própria semântica de sucesso e falha. Isso é muito mais sólido do que esconder mount operations em um método utilitário sem identidade.

Elemento de domínio	Natureza	Responsabilidade
<i>Container</i>	Agregado	Representa o ciclo de vida do container e referencia políticas e histórico de execução.
<i>RootSwitchPolicy</i>	Objeto de valor	Define estratégia preferida, fallback, política de cleanup e propagação.
<i>RuntimePlan</i>	Objeto de valor composto	Materializa a intenção declarativa em uma sequência ordenada e imutável de ações nativas.
<i>ExecutionAttempt</i>	Entidade	Registra uma tentativa concreta, com timestamps, PIDs, resultado e mountinfo.
<i>RootTransitionDomainService</i>	Serviço de domínio	Determina a estratégia semanticamente adequada para a transição de raiz.
<i>RootTransitionFailed</i>	Evento de domínio	Expõe falha rica, classificável e auditável ao resto do sistema.

8. Padrões de design relevantes (GoF)

A feature se beneficia muito de alguns padrões clássicos, desde que eles sejam usados para tornar o design mais explícito e não para ornamentar a implementação. O padrão mais importante é **Strategy**. A escolha entre *PutOldPivotStrategy*, *FDPivotStrategy* e *MoveRootChrootStrategy* é uma variação genuína de algoritmo sob o

mesmo contrato semântico. Cada estratégia deve receber um contexto comum, produzir um resultado comum e ser selecionada por política e capacidade.

O padrão **Template Method** ajuda a evitar duplicação sem apagar as diferenças entre estratégias. Há um esqueleto fixo para a operação de root transition: validar precondições, normalizar propagação, preparar o rootfs, executar a troca, corrigir o cwd, limpar oldroot, emitir telemetria e devolver resultado. O que varia é o miolo do passo “executar a troca” e parte da limpeza posterior. Um método template no C++ mantém esse esqueleto coeso e reduz o risco de uma estratégia esquecer etapas mandatórias, como `chdir("/")`.

O padrão **Command** é útil para modelar ações nativas auditáveis, como `BindMountRootfs`, `NormalizePropagation`, `PivotRootCommand`, `DetachOldRoot` e `MountProcFs`. Não se trata necessariamente de construir uma hierarquia excessiva de classes, mas de dar nome a unidades reversíveis ou pelo menos observáveis de trabalho. Isso simplifica logging, testes e eventual rollback local.

O padrão **State** se encaixa no agregado de lifecycle. Um container em *RootTransitionPending* aceita certos comandos e recusa outros; um container em *Running* não pode voltar para *RootfsStaged*; um container em *Failed* pode aceitar cleanup, mas não start sem novo attempt. Implementar explicitamente essa máquina de estados no plano de controle evita que a semântica do kernel vaze como ifs espalhados no código Python.

O padrão **Abstract Factory** também é valioso quando o projeto amadurecer para múltiplos perfis de execução. Um runtime rootful tradicional, um runtime com fallback sem pivot e um futuro runtime rootless provavelmente compartilharão parte do contrato, mas não toda a implementação. Uma fábrica de famílias de componentes permite produzir conjuntos coerentes de *NamespacePreparer*, *RootTransitionStrategy* e *StandardMountPlan* sem poluir o resto do sistema com condicionais por ambiente.

Na fronteira entre linguagens, os padrões **Facade** e **Adapter** são quase obrigatórios. O módulo Cython deve oferecer uma fachada estreita para o mundo Python, enquanto adapta tipos Python para estruturas nativas do C++. A regra saudável é simples: Python não conhece detalhes internos do C++; C++ não conhece a riqueza dinâmica dos objetos Python. O Cython faz o encaixe sem adquirir lógica de domínio própria.

Por fim, o padrão **Builder** é muito útil para montar o *RuntimePlan*. Um plano de runtime reúne rootfs, mounts ordenados, política de propagação, política de root switch, init command, stdio, environment e telemetry options. Um builder declarativo no lado Python facilita validações compostas e produz um DTO final imutável, ideal para serialização pela ACL em Cython.

Padrão	Aplicação na feature	Benefício concreto
Strategy	Seleção entre <i>pivot</i> clássico, <i>pivot</i> por FD e fallback move-root/chroot.	Permite variar algoritmo sem variar contrato observável.
Template Method	Esqueleto fixo da transição de raiz.	Evita esquecimento de etapas mandatórias e centraliza invariantes.
Command	Ações nativas individuais e auditáveis.	Melhora logging, testes e rollback local.
State	Estados do lifecycle do container.	Formaliza transições válidas e invalida operações incoerentes.

Padrão	Aplicação na feature	Benefício concreto
Abstract Factory	Famílias de runtime por perfil de execução.	Escala para perfis rootful, fallback e futuros ambientes rootless.
Facade / Adapter	Fronteira Cython entre Python e C++.	Estabiliza ABI e desacopla o domínio de detalhes de runtime.
Builder	Construção do <i>RuntimePlan</i> .	Produz planos imutáveis, validados e fáceis de serializar.

9. Requirements funcionais e não funcionais

A seguir, os requisitos são apresentados de forma textual e verificável. O objetivo não é apenas listar desejos, mas descrever comportamentos observáveis que garantam que a feature cobre, de fato, os conceitos de *pivot_root*, *switch_root*, cleanup de oldroot, correção do cwd, propagação de mounts e rastreabilidade operacional.

ID	Requirement funcional	Racional	Verificação
RF-01	O runtime deve validar, no lado nativo, que o rootfs existe, é diretório e corresponde ao caminho resolvido pelo plano de controle.	Evita confundir erro de staging com erro de root switch.	Teste de integração e erro estruturado.
RF-02	O runtime deve garantir que o rootfs seja mountpoint antes de tentar <i>pivot_root</i> .	Pré-condição exigida pelo kernel.	Inspeção de mountinfo antes do pivot.
RF-03	O runtime deve normalizar a propagação do mount pai do rootfs para evitar falha ou vazamento de mounts.	Compatibilidade com hosts em regime shared.	Teste em ambiente systemd/shared.
RF-04	O sistema deve suportar pelo menos uma estratégia explícita baseada em <i>pivot_root</i> e uma estratégia de fallback sem <i>pivot_root</i> .	Cobre caminho primário e contingência.	Testes por matriz de estratégia.
RF-05	Após a transição, o runtime deve sempre executar <code>chdir("/")</code> e expor esse fato na telemetria da operação.	Garante que o cwd não retenha o oldroot.	Teste com <code>pwd</code> e mountinfo.
RF-06	O sistema deve desmontar ou destacar o oldroot conforme política explícita de cleanup e registrar o resultado.	A feature só se completa com oldroot cleanup.	Teste de acessibilidade de oldroot.
RF-07	O runtime deve suportar rootfs readonly sem exigir diretórios auxiliares mutáveis no rootfs final.	Necessário para ambientes endurecidos.	Teste com rootfs somente leitura.

ID	Requirement funcional	Racional	Verificação
RF-08	O runtime deve montar ou mover pseudo-filesystems padrão após a troca de raiz, segundo um <i>StandardMountPlan</i> ordenado.	Evita acoplamento entre troca de raiz e mounts padrão.	Teste de bootstrap do init.
RF-09	O caminho pós-fork deve ser executado integralmente em código nativo, sem retorno ao interpretador Python no processo filho.	Segurança operacional e previsibilidade.	Revisão arquitetural e testes de processo.
RF-10	O sistema deve retornar mensagens estruturadas contendo estágio lógico, syscall, erro e snapshots relevantes quando houver falha.	Diagnóstico e auditabilidade.	Contrato de API e teste negativo.
RF-11	O plano de controle deve persistir a estratégia pretendida e a estratégia efetivamente usada em cada tentativa de execução.	Importante para auditoria e análise de fallback.	Inspeção de metadados persistidos.
RF-12	O runtime deve distinguir falhas de preparação, falhas de root transition e falhas pós-transition antes do <code>execve</code> .	Classificação operacional correta.	Testes de falha induzida por estágio.

ID	Requirement não funcional	Racional	Métrica ou evidência
RNF-01	A feature deve preservar isolamento de filesystem, sem mounts vazando do container para o host por propagação indevida.	Segurança e correção.	Testes de propagação em namespaces pareados.
RNF-02	O caminho crítico deve ser determinístico na ordem de syscalls e produzir logs estáveis por estágio.	Reprodutibilidade.	Traços de execução e testes repetidos.
RNF-03	O subsistema deve ser observável por mountinfo before/after, duração e estratégia efetiva.	Operação e suporte.	Presença dos campos em telemetria.
RNF-04	Descritores temporários e canais de sincronização devem usar <code>CLOEXEC</code> e ser fechados antes do <code>execve</code> .	Higiene de recursos.	Auditoria de FDs e testes de leak.
RNF-05	O contrato Python/Cython/C++ deve permanecer estável e versionável, com DTOs explícitos e enums congelados.	Manutenibilidade.	Versionamento do schema de bridge.
RNF-06	O runtime deve operar corretamente em hosts cujo mount tree inicial esteja em regime shared por padrão.	Compatibilidade com distribuições reais.	Teste em ambientes systemd.

ID	Requirement não funcional	Racional	Métrica ou evidência
RNF-07	O caminho de fallback deve ser claramente identificável e nunca acionado silenciosamente contra política do plano de controle.	Governança de comportamento.	Eventos e contrato de API.
RNF-08	O subsistema deve ser extensível para novas APIs de mount e futuros perfis rootless sem ruptura conceitual.	Evolução técnica.	Arquitetura por estratégia e factory.
RNF-09	O tempo de execução do root switch deve ser baixo e proporcional ao número de mounts relevantes, sem algoritmos de varredura desnecessários no caminho feliz.	Eficiência.	Benchmark de start e profiling.
RNF-10	A documentação interna deve tornar inequívoco o significado de cada estratégia e de cada política de cleanup.	Transferência de conhecimento.	Playbook e design docs aprovados.
RNF-11	Falhas pós-pivot e pré-exec não podem deixar processos órfãos semi-inicializados em execução.	Segurança operacional.	Testes de falha induzida e watchdog.
RNF-12	O subsistema deve ser suficientemente modular para permitir testes unitários em C++ e testes de integração orquestrados por Python.	Qualidade contínua.	Cobertura automatizada por camada.

10. Plano incremental de implementação

A melhor forma de chegar a uma feature robusta é não começar pelo caminho mais compacto do *runc*, e sim por um percurso incremental que maximize entendimento e validação. A Fase 1 deve entregar um runtime rootful mínimo com mount namespace isolado, normalização de propagação, bind mount do rootfs em si mesmo e *PutOldPivotStrategy*. Essa fase já precisa ter telemetria rica, snapshots de mountinfo e testes de cleanup. O objetivo não é imitar o produto final; é construir a gramática correta do problema.

A Fase 2 deve promover a *FDPIVOTStrategy* a estratégia principal. Aqui entram suporte nativo a readonly rootfs, wrappers de descritor mais maduros, política configurável de cleanup e conversão sistemática de erros em mensagens estruturadas. É também o momento de consolidar a máquina de estados no plano de controle, distinguindo claramente *RootTransitionApplied* de *Running*.

A Fase 3 introduz o fallback endurecido *MoveRootChrootStrategy* e um *StandardMountPlan* formal para pseudo-filesystems. O time só deve entrar nessa fase depois que o caminho primário estiver sólido, porque o fallback exige entendimento mais fino de mounts residuais, mascaramento de `/proc` e `/sys` e diferenças semânticas entre pivot e no-pivot.

A Fase 4 é de refinamento e futuro. Aqui entram adoção gradual das APIs modernas de mount do kernel, eventual suporte rootless, integração com overlays mais sofisticados, profiling e endurecimentos adicionais de

segurança. O objetivo não é reabrir a semântica do subsistema, mas apenas trocar a infraestrutura por mecanismos mais seguros ou mais eficientes.

Fase	Entrega principal	Capacidades	Critério de aceite
1	MVP com <i>PutOldPivotStrategy</i>	Mount namespace, propagação segura, bind mount do rootfs, <i>pivot_root</i> clássico, cleanup explícito e telemetria básica.	Container sobe em rootfs gravável e oldroot deixa de ser alcançável.
2	Padrão de produção com <i>FDPivotStrategy</i>	Suporte a readonly rootfs, wrappers RAII de FD, mensagens de erro ricas, machine state persistida.	Readonly rootfs funciona sem diretório auxiliar e com telemetria completa.
3	Fallback endurecido e mounts padrão	<code>MS_MOVE + chroot</code> , mascaramento de mounts perigosos, <i>StandardMountPlan</i> independente da estratégia.	Fallback passa na matriz de segurança e não é usado silenciosamente.
4	Modernização e expansão	APIs modernas de mount, ensaios para rootless, profiling, hardening e documentação final.	Sem ruptura de contrato e com ganhos mensuráveis de segurança ou ergonomia.

Estratégia de entrega

Não tente “ficar igual ao runc” no dia um. Primeiro faça o problema ficar visível, verificável e semanticamente correto. Depois substitua a estratégia de MVP pela estratégia de produção. Isso reduz risco técnico e acelera o aprendizado da equipe.

11. Estratégia de testes e validação

A feature é sensível o bastante para exigir testes em três níveis. O primeiro é unitário, no C++, cobrindo parse de mountinfo, escolha de estratégia, política de cleanup, wrappers de descritor e validação de pré-condições. O segundo é de integração nativa, executando namespaces reais e rootfs de teste, preferencialmente com BusyBox ou Alpine mínimos. O terceiro é de sistema, orquestrado pelo plano de controle em Python, validando persistência de estados, eventos, retries e semântica observável da API.

Os testes de integração precisam observar mais do que exit code. Antes e depois do root switch, o runtime deve coletar e armazenar `/proc/self/mountinfo`. O processo init de teste deve imprimir `pwd`, o conteúdo de `/proc/self/mountinfo` e resultados de `stat("/")`. Esses artefatos permitem comprovar que a nova barra é a esperada, que o cwd foi corrigido e que o oldroot não permanece visível no namespace.

Uma classe de teste indispensável é a de propagação. Em muitos hosts modernos, o mount tree inicial vem em modo *shared*. O runtime deve ser exercitado nesse ambiente para demonstrar que normaliza corretamente a propagação e não vaza bind mounts do rootfs de volta para o namespace pai. Outra classe indispensável é a de rootfs readonly, que valida a superioridade prática da estratégia por FD.

Também é importante induzir falhas em estágios específicos. Deve haver testes que forçam falha na normalização de propagação, falha no bind mount do rootfs, falha no `pivot_root`, falha no unmount do oldroot e falha na montagem posterior de `/proc`. Em cada caso, o sistema deve devolver a classificação correta do erro, sem deixar processo órfão ou mount residual incoerente.

Cenário de teste	O que precisa ser verificado	Sinal esperado
Host com mounts shared	O rootfs é bind-mounted sem vaziar para o namespace pai e o <code>pivot_root</code> não falha por propagação.	Mountinfo do host sem mount extra; start bem-sucedido.
Rootfs readonly	A estratégia por FD funciona sem diretório auxiliar mutável.	Container sobe e oldroot é desmontado.
Falha artificial em <code>pivot_root</code>	Erro volta com estágio correto, erro e snapshots úteis.	<i>RootTransitionFailed</i> com classificação PIVOT.
Modo estrito de unmount	O runtime tenta unmount normal antes do lazy detach e sinaliza fallback.	Evento indicando strict-failed-then-lazy ou strict-success.
Fallback <code>MS_MOVE</code> + <code>chroot</code>	Mounts perigosos fora do rootfs são mascarados ou desmontados antes do <code>chroot</code> .	<code>/proc</code> e <code>/sys</code> corretos após start.
Crash pós-pivot e pré-exec	Não há processo órfão semi-inicializado e o pai recebe diagnóstico completo.	Attempt marcado como failed com cleanup local executado.

Uma prática valiosa é incluir testes de regressão por diffs de mountinfo. Em vez de verificar apenas “subiu ou não subiu”, compare snapshots normalizados da tabela de mounts. Isso torna visíveis vazamentos de mount, paths residuais de oldroot e divergências entre estratégias. Essa forma de teste conversa muito bem com CI e com uma biblioteca de fixtures de rootfs mínimos.

12. Conclusão

A feature de `pivot_root` / `switch_root` em um Docker-like system from scratch só parece pequena quando descrita em poucas palavras. Na prática, ela é uma dobradiça entre o mundo declarativo do plano de controle e a materialidade dos syscalls do plano de dados. Uma implementação séria precisa articular mount namespace, propagação, mountpoints, cwd, cleanup de oldroot, pseudo-file systems, canais de erro, estado persistido e política explícita.

No contexto do seu projeto, a melhor direção é clara. Python deve continuar sendo o centro do control plane, dos metadados e da API. Cython deve ser uma fronteira estreita, previsível e versionável. C++ deve assumir toda a região onde o kernel pode punir abstrações frouxas: fork, namespace, mount, `pivot_root`, unmount, exec e higiene de descritores. Dentro desse desenho, a feature deve se chamar algo como *Root Transition* e ter estratégia de MVP pedagógica e estratégia de produção alinhada ao estado da prática.

Em termos concretos, a recomendação final é implementar primeiro uma *PutOldPivotStrategy* muito bem instrumentada, validá-la até que a equipe domine o problema e, em seguida, promover a *FDPivotStrategy* inspirada no *runc* a padrão de produção. O fallback `MS_MOVE + chroot` deve existir, mas como modo controlado e endurecido, nunca como atalho invisível. *switch_root* deve ser tratado como referência conceitual e como modo especializado para contextos de boot, não como semântica padrão de containers.

Síntese final

Se o sistema consegue provar, por telemetria e testes, que o rootfs virou a nova barra, que o cwd passou a ser `/`, que o oldroot deixou de ser alcançável e que nenhuma propagação indevida ocorreu, então a feature está correta. Todo o resto - estilo de implementação, nome das classes e detalhes do bridge - deve servir a essa prova.

Apêndice A. Sequências de syscall sugeridas

As sequências abaixo não pretendem ser drop-in code, mas sintetizam a ordem operacional sugerida para cada modo. Elas são úteis como especificação executável de baixo nível e como base para testes de regressão.

A.1 Sequência canônica para o caminho de produção

```
// C++ pseudocode - caminho de produção
validate_rootfs();
setup_sync_pipe();
enter_mount_namespace_or_clone();
set_root_propagation(MS_SLAVE | MS_REC);           // ou política configurada
make_rootfs_parent_private(rootfs);
ensure_rootfs_is_mountpoint(rootfs);               // bind-mount em si mesmo
oldroot_fd = open("/", O_DIRECTORY | O_RDONLY | O_CLOEXEC);
newroot_fd = open(rootfs, O_DIRECTORY | O_RDONLY | O_CLOEXEC);
fchdir(newroot_fd);
pivot_root(".", ".");
fchdir(oldroot_fd);
mount("", ".", "", MS_SLAVE | MS_REC, "");
umount2(".", cleanup_mode == LAZY ? MNT_DETACH : 0);
chdir("/");
apply_standard_mount_plan();
report_success_over_pipe();
execve(init_path, argv, envp);
```

A.2 Sequência de MVP com put_old explícito

```
validate_rootfs();
enter_mount_namespace_or_clone();
set_root_propagation(MS_SLAVE | MS_REC);
make_rootfs_parent_private(rootfs);
ensure_rootfs_is_mountpoint(rootfs);
mkdirat(rootfs_fd, ".pivot_old", 0700);           // reservado ao runtime
fchdir(rootfs_fd);
pivot_root(".", "./.pivot_old");
chdir("/");
mount("", "./.pivot_old", "", MS_SLAVE | MS_REC, "");
umount2("./.pivot_old", cleanup_mode == LAZY ? MNT_DETACH : 0);
rmdir("./.pivot_old");
apply_standard_mount_plan();
execve(init_path, argv, envp);
```


A.3 Sequência de fallback endurecido

```
validate_rootfs();
enter_mount_namespace_or_clone();
set_root_propagation(MS_SLAVE | MS_REC);
make_rootfs_parent_private(rootfs);
ensure_rootfs_is_mountpoint(rootfs);
detach_or_mask_external_procfs_and_sysfs();
mount(rootfs, "/", NULL, MS_MOVE, NULL);
chroot(".");
chdir("/");
apply_standard_mount_plan();
execve(init_path, argv, envp);
```

Apêndice B. Interfaces indicativas entre Python, Cython e C++

As interfaces abaixo ilustram como manter a fronteira entre domínio e execução nativa estreita e estável. O objetivo é expor semântica suficiente sem permitir que detalhes internos do runtime vazem para o plano de controle.

```
# Python - domínio / aplicação
@dataclass(frozen=True)
class RootSwitchPolicy:
    preferred_mode: str          # "pivot_fd", "pivot_put_old", "move_chroot"
    allow_fallback: bool
    cleanup_mode: str           # "strict" | "lazy"
    root_propagation: str       # "slave" | "private"
    readonly_rootfs: bool

@dataclass(frozen=True)
class RuntimePlan:
    container_id: str
    rootfs_path: str
    init_argv: tuple[str, ...]
    env: tuple[tuple[str, str], ...]
    mounts: tuple[dict, ...]
    root_switch_policy: RootSwitchPolicy
```

```
# Cython - ACL
cdef class NativeRuntimeBridge:
    cpdef NativeExecutionResult start_container(self, RuntimePlan plan)
```

```
// C++ - execução nativa
struct RootSwitchPolicyDto {
    Mode preferred_mode;
```

```

    bool allow_fallback;
    CleanupMode cleanup_mode;
    PropagationMode propagation;
    bool readonly_rootfs;
};

struct RuntimePlanDto {
    std::string container_id;
    std::string rootfs_path;
    std::vector<std::string> init_argv;
    std::vector<std::pair<std::string, std::string>> env;
    std::vector<MountDto> mounts;
    RootSwitchPolicyDto root_switch_policy;
};

struct ExecutionResultDto {
    bool success;
    Stage failed_stage;
    int errno_code;
    std::string errno_name;
    std::string message;
    std::string strategy_used;
    std::string mountinfo_before;
    std::string mountinfo_after;
};

```

Essa fronteira deixa claro que Python decide e persiste; Cython traduz; C++ executa. Também deixa claro que o contrato de retorno é estruturado o bastante para telemetria e suporte.

Referências

1. Michael Kerrisk. *pivot_root(2) - Linux manual page*. man7.org. Disponível em: https://man7.org/linux/man-pages/man2/pivot_root.2.html. Acesso em 12 mar. 2026.
2. util-linux project. *switch_root(8) - Linux manual page*. man7.org. Disponível em: https://man7.org/linux/man-pages/man8/switch_root.8.html. Acesso em 12 mar. 2026.
3. Michael Kerrisk. *mount_namespaces(7) - Linux manual page*. man7.org. Disponível em: https://man7.org/linux/man-pages/man7/mount_namespaces.7.html. Acesso em 12 mar. 2026.
4. Linux Kernel Documentation. *shared subtree operations*. Disponível em: <https://docs.kernel.org/filesystems/sharedsubtree.html>. Acesso em 12 mar. 2026.
5. Open Container Initiative. *Runtime Specification - config.md*. Disponível em: <https://github.com/opencontainers/runtime-spec/blob/main/config.md>. Acesso em 12 mar. 2026.
6. Solomon Hykes. *Introducing Docker 0.9*. Docker Blog, 2014. Disponível em: <https://www.docker.com/blog/docker-0-9-introducing-execution-drivers-and-libcontainer/>. Acesso em 12 mar. 2026.
7. IBM. *A history of low-level container runtimes*. Disponível em: <https://developer.ibm.com/articles/l-container-runtimes/>. Acesso em 12 mar. 2026.
8. opencontainers/runc. *libcontainer README*. Disponível em: <https://github.com/opencontainers/runc/tree/main/libcontainer>. Acesso em 12 mar. 2026.
9. opencontainers/runc. *libcontainer/rootfs_linux.go*. Disponível em: https://github.com/opencontainers/runc/blob/main/libcontainer/rootfs_linux.go. Acesso em 12 mar. 2026.
10. util-linux. *sys-utils/switch_root.c*. Disponível em: https://github.com/util-linux/util-linux/blob/master/sys-utils/switch_root.c. Acesso em 12 mar. 2026.
11. Red Hat. *Building a container by hand using namespaces: The mount namespace*. Disponível em: <https://www.redhat.com/en/blog/mount-namespaces>. Acesso em 12 mar. 2026.
12. Michael Kerrisk. *chroot(2) - Linux manual page*. man7.org. Disponível em: <https://man7.org/linux/man-pages/man2/chroot.2.html>. Acesso em 12 mar. 2026.